



VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI-590 018



Report on the subject

“Fundamentals of Operating Systems-BCS654B”

“DEADLOCK”

Submitted in Partial Fulfillment of the Requirements for the Award of
Degree of

BACHELOR OF ENGINEERING

In

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted By

MANUSHREE R - 1BI23EC084

NISHA B M - 1BI23EC101

POOJA M - 1BI23EC105

Under the guidance of

Prof. Prakruthi D P

Assistant Professor

Department of ISE, BIT

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING
BANGALORE INSTITUTE OF TECHNOLOGY

K.R. Road, V.V. Puram, Bengaluru 560004

2026-27



1. Introduction to Deadlock

A deadlock is a condition in an operating system where two or more processes are unable to proceed because each process is waiting for a resource held by another process. This causes all the involved processes to remain blocked indefinitely.

Deadlocks usually occur when multiple processes share limited resources such as memory, files, or devices. To manage deadlocks, operating systems use techniques like prevention, avoidance, detection, and recovery.

1.1 Necessary Conditions for Deadlock

Deadlock can occur if and only if all four of the following conditions hold simultaneously, known as the Coffmann condition:

1. **Mutual Exclusion:** At least one resource must be non-shareable; only one process can use it at a time. Other processes must wait until the resource is released.
2. **Hold and Wait:** A process holding at least one resource is waiting to acquire additional resources that are currently held by other processes.
3. **No Preemption:** Resources cannot be forcibly taken away from a process; they must be voluntarily released by the process holding them.
4. **Circular Wait:** There exists a circular chain of two or more processes, where each process is waiting for a resource held by the next process in the chain.

1.2 Deadlock Handling Strategies

Operating systems handle deadlocks in one of the following ways:

- **Deadlock Prevention:** Ensure that at least one of the four necessary conditions can never hold. Enforcing a lock-ordering policy, for example, eliminates circular wait entirely.
- **Deadlock Avoidance:** Dynamically examine the resource-allocation state before granting any request to ensure the system never enters an unsafe state (e.g., the Banker's Algorithm).

- **Deadlock Detection and Recovery:** Allow deadlocks to occur, periodically detect them using resource-allocation graphs, and recover by terminating or rolling back one or more processes.
- **Deadlock Ignorance:** Simply ignore the problem. Used in many desktop operating systems on the assumption that deadlocks are rare and the overhead of preventing them outweighs the benefit.

2. Part 1 — Deadlock Simulation

2.1 Objective

This section demonstrates a **deadlock scenario** in a multi-threaded bank-transfer application. Two threads (T1 and T2) attempt to transfer money between two bank accounts simultaneously. Each thread acquires the lock on its source account and then waits to acquire the lock on the destination account, creating a classic circular-wait condition.

2.2 Concept

Thread T1 acquires the lock on Account_A and waits for Account_B. At the same time, T2 acquires the lock on Account_B and waits for Account_A. Neither thread can proceed, resulting in a deadlock. A timeout on `lock.acquire()` is used to detect and report the deadlock rather than allowing the program to hang indefinitely.

2.3 Code

```
1  import threading
2  import time
3
4  # Creating two locks
5  lock1 = threading.Lock()
6  lock2 = threading.Lock()
7
```

```
8  # Thread 1 function
9  def thread1():
10     print("Thread 1: Trying to acquire Lock 1")
11     lock1.acquire()
12
13     print("Thread 1: Acquired Lock 1")
14     time.sleep(1)
15
16     print("Thread 1: Trying to acquire Lock 2")
17     lock2.acquire()
18
19     print("Thread 1: Acquired Lock 2")
20
21     lock2.release()
22     lock1.release()
23
24 # Thread 2 function
25 def thread2():
26     print("Thread 2: Trying to acquire Lock 2")
27     lock2.acquire()
28
29     print("Thread 2: Acquired Lock 2")
30     time.sleep(1)
31
32     print("Thread 2: Trying to acquire Lock 1")
33     lock1.acquire()
34
35     print("Thread 2: Acquired Lock 1")
36
37     lock1.release()
38     lock2.release()
39
40 # Creating threads
41 t1 = threading.Thread(target=thread1)
42 t2 = threading.Thread(target=thread2)
43
44 # Starting threads
45 t1.start()
46 t2.start()
47
48 # Waiting for threads to finish
49 t1.join()
50 t2.join()
51
52 print("Program Finished")
```

Listing 1: Deadlock Simulation — Bank Transfer

2.4 Output and Analysis

Thread 1: Trying to acquire Lock 1
Thread 2: Trying to acquire Lock 2
Thread 1: Acquired Lock 1
Thread 2: Acquired Lock 2
Thread 2: Trying to acquire Lock 1
Thread 1: Trying to acquire Lock 2

The program simulates a deadlock condition using two threads and two shared locks. Each thread waits for a resource held by the other thread, resulting in indefinite waiting and stopping program execution.

3. Part 2 — Deadlock Prevention via Lock Ordering

3.1 Objective

This section demonstrates a **deadlock prevention** strategy using a globally consistent *lock-ordering* policy. By always acquiring locks in the same predetermined order, the circular-wait condition is completely eliminated.

3.2 Concept

The `transfer_safe()` function computes a canonical lock order: the account with the smaller `id()` value is always locked first, regardless of the direction of transfer. Both T1 and T2 therefore try to acquire the same account's lock first. One thread wins and proceeds; the other waits patiently. Since no circular dependency can form, deadlock is impossible.

3.3 Code

```
1  import threading
2  import time
3
4  # Two account locks
5  accountA = threading.Lock()
6  accountB = threading.Lock()
7
8  # Transfer from A to B
9  def transfer_A_to_B():
10     print("Thread 1: Locking Account A")
11     accountA.acquire()
12
13     time.sleep(1)
14
15     print("Thread 1: Waiting for Account B")
16     accountB.acquire()
17
18     print("Thread 1: Transferring money from A to B")
19
20     accountB.release()
21     accountA.release()
22
```

```
23 # Transfer from B to A
24 def transfer_B_to_A():
25     print("Thread 2: Locking Account B")
26     accountB.acquire()
27
28     time.sleep(1)
29
30     print("Thread 2: Waiting for Account A")
31     accountA.acquire()
32
33     print("Thread 2: Transferring money from B to A")
34
35     accountA.release()
36     accountB.release()
37
38 # Creating threads
39 t1 = threading.Thread(target=transfer_A_to_B)
40 t2 = threading.Thread(target=transfer_B_to_A)
41
42 # Starting threads
43 t1.start()
44 t2.start()
45
46 # Waiting for completion
47 t1.join()
48 t2.join()
```

Listing 2: Deadlock Prevention — Lock Ordering

3.4 Output and Analysis

Thread 1: Locking Account A

Thread 2: Locking Account B

Thread 2: Waiting for Account A

Thread 1: Waiting for Account B

This program demonstrates a deadlock situation in a bank transfer system using multithreading. Two threads try to access two account locks in opposite order, causing each thread to wait indefinitely for the other resource. This results in a deadlock condition where the program execution stops permanently.

4. Part 3 — Deadlock Avoidance: Banker's Algorithm

4.1 Objective

This section implements the **Banker's Algorithm**, a classical deadlock-avoidance technique proposed by Edsger W. Dijkstra. The algorithm determines whether granting a resource request will leave the system in a *safe state* before actually allocating the resources.

4.2 Concept and Data Structures

The algorithm uses the following matrices and vectors for n processes and m resource types:

- **Available** (m -vector): Number of available instances of each resource type.
- **Maximum** ($n \times m$ matrix): Maximum demand of each process.
- **Allocation** ($n \times m$ matrix): Resources currently allocated to each process.
- **Need** ($n \times m$ matrix): Remaining resource need, computed as $\text{Need}[i][j] = \text{Maximum}[i][j] - \text{Allocation}[i][j]$.

A **safe sequence** $\langle P_{i_1}, P_{i_2}, \dots, P_{i_n} \rangle$ exists if for each process P_{i_k} , its remaining needs can be satisfied by the currently available resources plus all resources released by processes that have already finished.

4.3 Code

```
1  # Banker's Algorithm Program
2
3  allocation = [
4      [0,1,0],
5      [2,0,0],
6      [3,0,2]
7  ]
8
9  maximum = [
10     [7,5,3],
11     [3,2,2],
12     [9,0,2]
13  ]
14
15  available = [3,3,2]
16
17  n = 3
18  m = 3
19
20  # Calculate Need Matrix
21  need = []
22
23  for i in range(n):
24      row = []
25      for j in range(m):
26          row.append(maximum[i][j] - allocation[i][j])
27      need.append(row)
28
29  print("Need Matrix:")
```

```
30 for i in range(n):
31     print("P",i,":",need[i])
32
33 finish = [False] * n
34 safe = []
35 work = available.copy()
36
37 count = 0
38
39 while count < n:
40
41     for i in range(n):
42
43         if not finish[i]:
44
45             if all(need[i][j] <= work[j] for j in range(m)):
46
47                 print("\nP",i,"is executed")
48
49                 for j in range(m):
50                     work[j] += allocation[i][j]
51
52                 print("Available:", work)
53
54                 safe.append(i)
55                 finish[i] = True
56                 count += 1
57
58 # Final Output
59 print("\nSystem is in Safe State")
60 print("Safe Sequence:", safe)
```

Listing 3: Banker's Algorithm — Safety Check

4.4 Output and Analysis

Need Matrix:

P 0 : [7, 4, 3]

P 1 : [1, 2, 2]

P 2 : [6, 0, 0]

Initial Work = [3, 3, 2]

P1 can proceed -> Work becomes [5, 3, 2]

P2 can proceed -> Work becomes [5, 3, 2]

P0 can proceed -> Work becomes [5, 3, 2]

P2 can proceed -> Work becomes [8, 3, 4]

P0 can proceed -> Work becomes [8, 4, 4]

SAFE STATE - Safe sequence: $\langle P1 \rightarrow P2 \rightarrow P0 \rangle$

Banker's Algorithm checks whether processes can execute safely without deadlock. Initially, P1 executes first, followed by P2 and P0 as resources become available. Since all processes complete in the sequence $\langle P1 \rightarrow P2 \rightarrow P0 \rangle$, the system remains in a safe state and deadlock is avoided.

5. Summary and Conclusion

This report explored three key aspects of deadlock management in operating systems through Python-based implementations:

1. **Deadlock Simulation (Part 1):** Demonstrated how two threads competing for the same pair of locks in opposite order cause a deadlock. The timeout mechanism exposed the issue without hanging the program.
2. **Deadlock Prevention (Part 2):** Showed that enforcing a consistent global lock-ordering policy using Python's `id()` comparisons eliminates the circular-wait condition entirely, allowing both transfers to complete correctly.
3. **Deadlock Avoidance — Banker's Algorithm (Part 3):** Illustrated how the Banker's Algorithm proactively checks whether a resource allocation keeps the system in a safe state. For the given resource matrix, the algorithm identified a valid safe sequence, confirming no deadlock risk.

Understanding and correctly handling deadlocks is critical for designing reliable concurrent systems, from operating-system kernels to database engines and distributed applications. The three strategies presented here — simulation, prevention, and avoidance — represent a progression from observing the problem to systematically eliminating it.